รายงานย่อบทความ OmniLedger

ชื่อบทความ: OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding
ผู้เขียน: Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly และ Bryan Ford (École Polytechnique Fédérale de Lausanne, Switzerland) และ Ewa Syta (Trinity College, USA)
งานประชุม: 2018 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 20–24 พฤษภาคม 2018, หน้า 583–598
DOI: 10.1109/SP.2018.000-5 (IEEE Xplore 8418625)
ผู้จัดทำ: นายธีรภัทร ภู่ระย้า ลำดับที่ 10
รหัสนิสิต: 66362416
รายวิชา: 305432 เครือข่ายเซนเซอร์ (Assignment Paper 3: บทความที่เกี่ยวข้องกับโครงงานวิศวกรรมคอมพิวเตอร์)

1 บทคัดย่อฉบับแปลและสรุป (Abstract Translation and Executive Summary)

บัญชีแยกประเภทแบบกระจายศูนย์ (Distributed Ledger หรือ DL) ที่ใช้ใน blockchain แบบ permissionless ยังประมวลผลธุรกรรมได้น้อยกว่าระบบชำระเงินแบบรวมศูนย์อย่าง Visa มาก สาเหตุหลักคือระบบส่วนใหญ่ **ขยายขนาดแบบ scale-out ไม่ได้** การเพิ่มจำนวน validator ไม่ได้ทำให้ความสามารถรวมเพิ่มขึ้น เพราะ validator ทุกตัวต้องตรวจทุกธุรกรรม ส่วนระบบที่ขยายได้ก็ต้องแลกกับความปลอดภัยหรือความเป็นกระจายศูนย์ ผู้เขียนสรุปข้อแลกเปลี่ยนนี้เป็นสามเหลี่ยมของ Decentralization, Scale-Out และ Security (Fig. 1 ของ [1]) ซึ่งระบบก่อนหน้าทำได้เพียงสองในสามด้าน

OmniLedger แบ่ง validator ออกเป็นกลุ่มย่อยเรียกว่า **shard** ให้แต่ละ shard ประมวลผลธุรกรรมในส่วนของตนแบบขนาน และรักษาความปลอดภัยด้วยกลไกสี่ส่วน ได้แก่ (1) สุ่มจัด validator เข้า shard ใหม่ทุก epoch ด้วยค่าสุ่มที่ผู้โจมตีบิดเบือนไม่ได้จากโปรโตคอล **RandHound** ร่วมกับการเลือกผู้นำด้วย **VRF** (2) ใช้ **ByzCoinX** ซึ่งเป็น Byzantine consensus ภายใน shard ที่ทนต่อโหนดล้มเหลวและ commit บล็อกแบบขนานได้ (3) ใช้ **Atomix** ให้ธุรกรรมที่เกี่ยวกับหลาย shard สำเร็จทั้งหมดหรือยกเลิกทั้งหมด และ (4) ใช้ **state blocks** ตัดประวัติบัญชีเก่า ร่วมกับ **trust-but-verify** เพื่อยืนยันธุรกรรมมูลค่าต่ำได้เร็ว ผู้เขียนพัฒนาต้นแบบด้วยภาษา Go

ผลทดลองบน DeterLab ที่จำลองโหนดได้สูงสุด 1,800 โหนดแสดงว่า throughput เพิ่มจาก **439 เป็น 5,850 ธุรกรรมต่อวินาที (tps)** เมื่อเพิ่ม shard จาก 1 เป็น 16 การยืนยันชั้นแรกใช้เวลา **1.38 วินาที** และโอกาสที่ระบบล้มเหลวต่อ epoch เท่ากับ $\mathbf{4 \times 10^{-5}}$ หรือประมาณ 1 ครั้งใน 68.5 ปีเมื่อ epoch ยาว 1 วัน ผู้เขียนสรุปว่าระบบรองรับภาระระดับ Visa ได้ อย่างไรก็ดี ผู้จัดทำพบว่าตัวเลข throughput ที่ 1,800 โหนดในแต่ละส่วนของบทความไม่ตรงกัน และภาคผนวกของผู้เขียนเองประเมินว่าเมื่อธุรกรรมส่วนใหญ่ต้องข้าม shard ความเร็วที่เพิ่มขึ้นจะต่ำกว่าผลในตารางมาก รายละเอียดอยู่ในหัวข้อ 6

2 ที่มาและปัญหาวิจัย (Motivation and Problem Statement)

ข้อแลกเปลี่ยนระหว่าง scale-out กับความปลอดภัยและความเป็นกระจายศูนย์: ใน Nakamoto consensus ของ Bitcoin ทุกโหนดเก็บและตรวจทุกธุรกรรม ทำให้เพิ่มโหนดแล้วระบบไม่เร็วขึ้น ผู้เขียนยกตัวเลขว่า blockchain ของ Bitcoin โตวันละประมาณ 144 MB และมีขนาดรวมประมาณ 133 GB ขณะที่ระบบที่รับภาระระดับ Visa (ประมาณ 4,000 tx/s ที่ 500 ไบต์ต่อธุรกรรม) จะสร้างข้อมูลได้มากกว่า 150 GB ต่อวัน งานก่อนหน้าแก้ปัญหาได้เพียงบางด้าน เช่น RSCoin [5], Elastico [4] และ ByzCoin [2] ดังตารางซึ่งสรุปจาก Table IV ของบทความ

ระบบ	Scale-Out	เวลาก่อนล้มเหลว (ประมาณการ)	Latency ที่วัดได้	ข้อจำกัดหลัก
RSCoin	ได้ เฉพาะแบบ permissioned	ไม่ระบุ	1 วินาที	ต้องพึ่งหน่วยงานกลาง
Elastico	ได้ บน Proof-of-Work	1 ชั่วโมง	800 วินาที	shard เล็ก ความปลอดภัยต่ำ
ByzCoin	ไม่ได้	19 ปี	40 วินาที	ทุกโหนดตรวจทุกธุรกรรม
Nakamoto (Bitcoin)	ไม่ได้	ไม่ระบุ	600 วินาที	ช้าและไม่ขยายตัว
OmniLedger	ได้	68.5 ปี	1.5 วินาที	ตามที่ผู้เขียนรายงาน

ความท้าทายหลัก 3 ประการตามที่ผู้เขียนระบุ (Section I):

1. **การเลือก validator ที่เป็นตัวแทนทางสถิติ:** ต้องเลือกกลุ่ม validator โดยทนต่อ Sybil attack บนพื้นฐานแบบ permissionless เช่น Proof-of-Work หรือ Proof-of-Stake
2. **การป้องกันไม่ให้ shard ใดถูกยึดตลอดอายุระบบ:** shard ต้องใหญ่พอ และต้องสุ่มสร้างใหม่เป็นระยะโดยผู้โจมตีบิดเบือนผลสุ่มไม่ได้ (bias-resistant)
3. **ความถูกต้องของธุรกรรมข้าม shard:** ธุรกรรมที่ใช้เงินจากหลาย shard ต้องสำเร็จหรือยกเลิกแบบ atomic เพื่อไม่ให้เงินถูกใช้ซ้ำหรือถูกล็อกค้างถาวร

แบบจำลองผู้โจมตี (Threat Model, Section III): validator ที่ไม่ซื่อสัตย์มีได้ไม่เกิน 25% ($n = 4f$) ผู้โจมตีปรับตัวได้ช้าในระดับ epoch (mildly adaptive) ช่องทางสื่อสารระหว่าง validator ที่ซื่อสัตย์เป็นแบบ synchronous และผู้โจมตีมีกำลังคำนวณจำกัดภายใต้สมมติฐานความยากของ computational Diffie-Hellman เป้าหมายของระบบคือไม่มีจุดล้มเหลวจุดเดียว, shard ทำงานถูกต้องต่อเนื่อง, ธุรกรรมเป็น atomic, throughput เพิ่มเชิงเส้นตามจำนวน validator, ใช้พื้นที่เก็บข้อมูลต่ำ และ latency ต่ำ

3 วิธีดำเนินการวิจัย (Proposed Methodology and Architecture)

OmniLedger ทำงานเป็นรอบเรียกว่า **epoch** (ในการทดลองยาว 1 วัน) validator ต้องลงทะเบียนใน identity blockchain ก่อนเริ่ม epoch ด้วยกลไกที่ทนต่อ Sybil attack จากนั้นระบบทำงานเป็น 4 ขั้นตอน (Fig. 2 ของบทความ)

1. **ขั้นที่ 1 การจัด validator เข้า shard ด้วยค่าสุ่มที่บิดเบือนไม่ได้ (Sharding via Bias-Resistant Randomness):** validator ทุกตัวคำนวณ ticket ด้วย Verifiable Random Function (VRF) เพื่อเลือกผู้นำ ผู้นำรัน RandHound [3] สร้างค่าสุ่มสาธารณะที่ตรวจสอบได้ แล้วใช้ค่าสุ่มนี้สร้าง permutation จัด validator เข้า shard ถ้าผู้นำไม่ซื่อสัตย์และไม่เผยค่าสุ่ม ระบบจะเลือกผู้นำใหม่ เพื่อไม่ให้ shard หยุดทำงานระหว่างเปลี่ยน epoch ระบบจะทยอยสลับ validator ครั้งละ k = log(n/m) ตัว ซึ่งน้อยกว่าหนึ่งในสามของขนาด shard (Section IV-B)
2. **ขั้นที่ 2 ฉันทามติภายใน shard ด้วย ByzCoinX:** ByzCoinX ปรับปรุงจาก ByzCoin [2] ซึ่งใช้ collective signing ผ่านต้นไม้การสื่อสาร โดยเปลี่ยนเป็นโครงสร้างแบบกลุ่มที่มีความลึกคงที่ (ผู้นำ → ผู้นำกลุ่ม → สมาชิก) ทำให้ทนต่อโหนดล้มเหลวได้ดีกว่า และวิเคราะห์ dependency ของธุรกรรม UTXO เพื่อ commit ธุรกรรมที่ไม่ขัดแย้งกันในบล็อกที่ต่างกันแบบขนาน
3. **ขั้นที่ 3 ธุรกรรมข้าม shard ด้วย Atomix:** เป็นโปรโตคอล atomic commit แบบ lock-then-unlock สามขั้น ได้แก่ **Initialize** (client ส่งธุรกรรมไปยัง input shard ทุกตัว), **Lock** (แต่ละ input shard ตรวจและล็อก UTXO แล้วออก proof-of-acceptance หรือ proof-of-rejection ซึ่งเป็น Merkle proof ที่มีลายเซ็นของ shard) และ **Unlock** (ถ้าได้ proof-of-acceptance ครบ client นำไป commit ที่ output shard ถ้ามี proof-of-rejection แม้หนึ่งใบ client นำไปขอคืน UTXO ที่ล็อกไว้) client เป็นผู้ขับเคลื่อนโปรโตคอล โดย shard ไม่ต้องคุยกันโดยตรง และฝ่ายอื่นทำแทนได้เมื่อ time-out (Section IV-C, Fig. 3)
4. **ขั้นที่ 4 ลดภาระการจัดเก็บและ latency: state blocks** คือสรุปสถานะ UTXO ของ shard ที่ออกเป็นระยะและมีลายเซ็นรวมของ shard validator ใหม่จึงดาวน์โหลดเฉพาะ state block ล่าสุดแทนการ replay ประวัติทั้งหมด ส่วน **trust-but-verify** ให้กลุ่ม optimistic validator ขนาดเล็กยืนยันธุรกรรมมูลค่าต่ำก่อน แล้วให้ core validator ของ shard ตรวจซ้ำภายหลังด้วยบล็อกขนาดใหญ่ (Section V, Fig. 4)

แบบจำลองความปลอดภัยเชิงความน่าจะเป็น (Section VI): ผู้เขียนระบุเองว่าการวิเคราะห์ในส่วนนี้เป็นแบบไม่เป็นทางการ (informal) โดยมีสมการหลักดังนี้

ความปลอดภัยของ shard หนึ่งตัว: shard ขนาด n โหนดจะปลอดภัยเมื่อมีโหนดไม่ซื่อสัตย์ไม่เกิน $\lfloor n/3 \rfloor$ เพราะ Byzantine consensus ต้องการ n = 3f + 1 [6], [7] เมื่อสมมติว่ามี validator จำนวนมากไม่จำกัด การสุ่มเข้า shard เป็นการแจกแจงทวินาม ผู้จัดทำเขียนใหม่ในรูปที่ถูกต้องดังนี้ (รูปที่พิมพ์ในบทความมีข้อผิดพลาด ดูหัวข้อ 6)

$$P[X \le \lfloor n/3 \rfloor] = \Sigma_{k=0}^{\lfloor n/3 \rfloor} C(n, k) \cdot m^k \cdot (1-m)^{n-k} \quad (1)$$

โดย X คือจำนวนโหนดไม่ซื่อสัตย์ใน shard, n คือขนาด shard, m คือสัดส่วนผู้โจมตี (สูงสุด 0.25) และ C(n, k) คือจำนวนวิธีเลือก k จาก n โอกาสที่ shard ล้มเหลว $P[X_S]$ คือส่วนเติมเต็มของค่านี้

ความล้มเหลวทั้ง epoch: ใช้ union bound รวมทุก shard และยอมให้ผู้โจมตีรัน RandHound ซ้ำได้ l ครั้ง

$$P[X_E] \le \Sigma_{k=0}^{l} (1/4^k) \cdot n \cdot P[X_S] \Rightarrow P[X_E] \le (4/3) \cdot n \cdot P[X_S] \text{ เมื่อ } l \to \infty \quad (2)$$

โดยในสมการนี้ n คือจำนวน shard (ไม่ใช่ขนาด shard) และ l คือจำนวนรอบที่ผู้โจมตีคุมผู้นำติดกัน ผลที่ผู้เขียนคำนวณคือ ผู้โจมตี 12.5% กับ 16 shard ได้ 4×10^{-5} ต่อ epoch หรือประมาณ 1 ครั้งใน 68.5 ปี ผู้จัดทำตรวจแล้วว่า $1 / (4 \times 10^{-5}) = 25{,}000$ วัน ≈ 68.5 ปี และเมื่อคำนวณซ้ำด้วยสมการ (1) ที่แก้ขอบบน

แล้ว (shard 70 โหนด) ได้ 2.2×10^{-6} ต่อ shard และ 4.6×10^{-5} ต่อ epoch ใกล้กับค่าที่รายงาน

โอกาสที่ผู้โจมตีคุมผู้นำติดกันและการตรวจกลุ่มสื่อสาร: $P[X \geq n] = 1/4^n$ ซึ่งต่ำกว่า 10^{-6} เมื่อ $n = 10$ ผู้โจมตีจึงคุม RandHound ได้ไม่เกิน 10 รอบติดกัน ส่วนการตรวจกลุ่มสื่อสารของ ByzCoinX ใช้ hypergeometric distribution กลุ่มขนาด 25 จาก 600 โหนด มีโอกาส 99.93% ที่มีโหนดไม่ซื่อสัตย์น้อยกว่า 15 และ union bound ของ 25 กลุ่มได้ 98.25%

4 การทดลองและผลลัพธ์ (Experiments and Quantitative Evaluation)

สภาพแวดล้อมการทดสอบ (Section VII และ VIII-A): ต้นแบบเขียนด้วยภาษา Go รันบน DeterLab ด้วยเครื่องจริง 60 เครื่อง (Intel E5-2420 v2, RAM 24 GB, เครือข่าย 10 Gbps) จำลองโหนดสูงสุด 1,800 โหนด จำกัดแบนด์วิดท์ทุกลิงก์ที่ 20 Mbps และหน่วงเวลา 100 ms เพื่อเลียนแบบเครือข่ายกระจายทั่วโลก ข้อมูลตั้งต้นคือ 10,000 บล็อกแรกของ Bitcoin

ตารางที่ 1 สรุปตัวชี้วัดหลักจากบทความ

ตัวชี้วัด (Metric)	ค่าที่รายงาน	แหล่งในบทความ	ส่วนเบี่ยงเบนมาตรฐาน (SD)
Throughput เมื่อเพิ่ม shard จาก 1 เป็น 16 (shard ละ 70 โหนด, ผู้โจมตี 12.5%)	**439 → 5,850 tps**	Table II	ไม่รายงาน
เวลายืนยันชั้นแรก (trust-but-verify) ที่ผู้โจมตี 1% ถึง 12.5%	**1.38 วินาที**	Table I	ไม่รายงาน
เวลายืนยันชั้นสุดท้ายที่ผู้โจมตี 25% (shard 600 โหนด)	**62.96 วินาที**	Table I	ไม่รายงาน
โอกาสล้มเหลวต่อ epoch (ผู้โจมตี 12.5%, 16 shard)	**4×10^{-5}** (≈ 1 ครั้งใน 68.5 ปี)	Section VI-C	วิเคราะห์เชิงทฤษฎี
ประสิทธิภาพของ ByzCoinX เทียบ ByzCoin (4 บล็อกขนาน)	**throughput +20%, latency −35%**	Fig. 9, Table III	ไม่รายงาน
สัดส่วนเวลาของ RandHound ในการเปลี่ยน epoch	**มากกว่า 70%** (กรณีแย่สุด ประมาณ 3 ชั่วโมง)	Fig. 7	ไม่รายงาน
จุดคุ้มทุนของ state blocks	**validator ตามไม่ทันเกิน 19 วัน** (2,736 บล็อก Bitcoin)	Fig. 11	ไม่รายงาน

ตารางที่ 2 การขยายแบบ scale-out ตาม Table II ของบทความ (สองคอลัมน์ขวาผู้จัดทำคำนวณเพิ่ม)

จำนวน shard (m)	1	2	4	8	16
Throughput (tps)	439	869	1,674	3,240	5,850
อัตราเร่งเทียบ 1 shard	1.00×	1.98×	3.81×	7.38×	13.33×
ประสิทธิภาพเทียบเชิงเส้นอุดมคติ	100%	99.0%	95.3%	92.3%	83.3%

ตารางที่ 3 เวลายืนยันธุรกรรมตาม Table I ของบทความ (วินาที)

การตั้งค่า [ขนาด shard, สัดส่วนผู้โจมตี]	[4, 1%]	[25, 5%]	[70, 12.5%]	[600, 25%]
Regular validation	1.38	5.99	8.04	14.52
ยืนยันชั้นแรก (1st level)	1.38	1.38	1.38	4.48
ยืนยันชั้นที่สอง (2nd level)	1.38	55.89	41.84	62.96

การอ่านผล: throughput เพิ่มเกือบเชิงเส้น โดยที่ 16 shard ยังได้ประสิทธิภาพ 83.3% ของค่าอุดมคติ โหมด trust-but-verify ให้การยืนยันชั้นแรกเร็วมาก แต่ยอมรับโอกาสน้อยกว่า 10% ที่การยืนยันนั้นจะเสี่ยงต่อ double-spending จึงเหมาะกับธุรกรรมมูลค่าต่ำ ส่วนธุรกรรมมูลค่าสูงต้องรอชั้นที่สองซึ่งใช้เวลา 41.84 ถึง 62.96 วินาที บทความระบุว่า latency ที่ client เห็นเมื่อใช้ Atomix สูงกว่า consensus latency เกือบสองเท่า และไม่เพิ่มมากเมื่อจำนวน shard เพิ่ม ข้อนี้เห็นชัดใน Fig. 8 ที่ shard ขนาด 70 โหนด (ราว 9 ถึง 12 วินาทีตั้งแต่ 1 ถึง 7 shard) แต่ที่ shard ขนาด 600 โหนด ค่านี้เพิ่มจากราว 15 เป็น 26 วินาทีเมื่อเพิ่มจาก 1 เป็น 3 shard

ตัวเลขที่ 1,800 โหนด: บทนำรายงาน 6,000 tps ที่ latency ประมาณ 10 วินาที (ผู้โจมตี 12.5%) และ 2,250 tps ในโหมด trust-but-verify ที่ผู้โจมตี 25% ส่วน Section VIII-B รายงาน 13,000 tps เมื่อแบ่ง 1,800 โหนดเป็น 25 shard ที่ผู้โจมตี 12.5% และเรียกว่าประมาณ 3 เท่าของ Visa ผู้จัดทำอ่านค่าจากกราฟ Fig. 6 โดยคำนวณจากพิกัดเวกเตอร์ในไฟล์ PDF ได้ประมาณ 6,800 tps (regular) และ 19,900 tps (trust-but-verify) สำหรับการตั้งค่าเดียวกัน จึงไม่มีแท่งใดตรงกับ 13,000 tps

5 การมีส่วนร่วมทางวิชาการ (Academic Contributions)

1. **การแบ่ง shard แบบ scale-out ที่วิเคราะห์ความปลอดภัยได้เชิงปริมาณ:** บทความนำการสุ่มแบบบิดเบือนไม่ได้ (RandHound ร่วมกับ VRF) มาเป็นฐานของการจัด validator เข้า shard และผูกขนาด shard กับโอกาสล้มเหลวด้วย Eq. (1) และ Eq. (2) ทำให้ผู้ออกแบบเลือกขนาด shard จากระดับความเสี่ยงที่ยอมรับได้ (เช่น 10^{-6}) แทนการกำหนดตามความรู้สึก เป็นการแก้จุดอ่อนของ Elastico ที่ใช้ shard เล็กจนเวลาเฉลี่ยก่อนล้มเหลวเหลือเพียง 1 ชั่วโมง
2. **Atomix: atomic commit ข้าม shard ภายใต้สมมติฐาน Byzantine:** บทความปรับแนวคิด atomic commit แบบดั้งเดิมให้ใช้กับ shard ที่ทำงานด้วย BFT consensus โดยไม่ต้องมี coordinator กลางและไม่ต้องให้ shard คุยกันเอง แต่ใช้ proof ที่มีลายเซ็นของ shard เป็นหลักฐานในการ commit หรือ abort ธุรกรรมจึงสำเร็จทั้งหมดหรือคืนเงินที่ล็อกได้เสมอ
3. **การออกแบบเชิงระบบที่ทำให้ใช้งานได้จริงและวัดผลบนต้นแบบ:** ByzCoinX เพิ่มทั้งความทนทานและการทำงานขนาน, state blocks ลดภาระ bootstrap ของ validator ใหม่ และ trust-but-verify แยกเส้นทางธุรกรรมตามระดับความเสี่ยง ทั้งหมดถูกพัฒนาเป็นต้นแบบภาษา Go และวัดผลที่ 1,800 โหนด ซึ่งมากกว่าการทดลองของ Elastico (1,600) และ ByzCoin (1,008) ตามที่ Table IV ของบทความสรุปไว้

6 การคิดเชิงวิพากษ์: จุดเด่น และจุดด้อย/ข้อจำกัด (Critical Evaluation)

จุดเด่น 5 ประการ

1. **ออกแบบครบทั้งระบบ:** แก้ครบทั้งการจัด shard, consensus, ธุรกรรมข้าม shard, การจัดเก็บ และ latency ในสถาปัตยกรรมเดียว ไม่ได้แก้เพียงส่วนเดียว

2. **ความปลอดภัยวัดได้เป็นตัวเลข:** การผูกขนาด shard กับโอกาสล้มเหลวทำให้ตรวจสอบและปรับค่าพารามิเตอร์ได้อย่างมีเหตุผล
3. **มีต้นแบบจริงและทดลองขนาดใหญ่:** ต้นแบบภาษา Go ทดสอบที่ 1,800 โหนดด้วยข้อมูลธุรกรรมจริงจาก Bitcoin
4. **ผู้เขียนเปิดเผยข้อจำกัดของตนเอง:** Section X และ Appendix C ยอมรับว่า throughput ขึ้นกับรูปแบบธุรกรรม และการเปลี่ยน epoch มีต้นทุนสูง
5. **ผลกระทบต่อวงการสูง:** IEEE Xplore แสดงว่าบทความถูกอ้างอิง 1,078 ครั้ง (ณ 26 กันยายน 2026) สูงที่สุดในกลุ่มบทความที่ผู้จัดทำคัดเลือก

จุดด้อยและข้อจำกัด 6 ประการ

1. **ผล scale-out ไม่ได้แยกผลของธุรกรรมข้าม shard:** Table II ให้อัตราเร่ง 3.81 เท่าที่ 4 shard แต่ Appendix C (หน้า 597) ของผู้เขียนประเมินเองว่าเมื่อกระจาย UTXO แบบสุ่มตาม hash ธุรกรรมส่วนใหญ่จะข้าม shard และอัตราเร่งจาก 1 ไป 4 shard จะเหลือประมาณ **1.56 เท่า** ผู้จัดทำคำนวณซ้ำแบบนับงานรวมต่อธุรกรรม (m / E[k]) ได้เพียง 1.46 ถึง 1.73 เท่า เพราะสูตรของผู้เขียนเฉลี่ยค่า 1/k ซึ่งให้ค่าสูงกว่าเสมอตามอสมการของ Jensen บทความไม่ระบุสัดส่วนธุรกรรมข้าม shard ในการทดลอง throughput และไม่มีการทดลองที่ปรับสัดส่วนนี้
2. **สมการและตัวเลขในส่วนคณิตศาสตร์มีข้อผิดพลาด:** Eq. (1) ที่พิมพ์ (หน้า 591) ใช้ขอบบนของผลรวมเป็น n ซึ่งทำให้ผลรวมเท่ากับ 1 เสมอตามทฤษฎีบททวินาม ขอบบนที่ถูกต้องคือ $\lfloor n/3 \rfloor$ สูตรเวียนเกิด Eq. (3) ใน Appendix C ใช้สัมประสิทธิ์ $(m - k)/m$ ซึ่งทำให้ผลรวมความน่าจะเป็นไม่เท่ากับ 1 (ค่าที่ถูกคือ $(m - k + 1)/m$) ค่า 3.7% ที่อ้างควรเป็นประมาณ 11.1% และค่า 1.56 เท่าตรงกับกรณี 4 UTXO ไม่ใช่ 3 UTXO ตามที่ข้อความระบุ นอกจากนี้สัญลักษณ์ n และ m ถูกใช้ต่างความหมายในแต่ละส่วน (ขนาด shard, จำนวน shard และสัดส่วนผู้โจมตี) และข้อความในหัวข้อ VI-B เรียกด้านซ้ายขวาของ Fig. 5 สลับกับคำบรรยายใต้รูป
3. **ตัวเลขพาดหัวไม่สอดคล้องกัน:** throughput ที่ 1,800 โหนดมีทั้ง 6,000 tps (บทนำ) และ 13,000 tps (Section VIII-B) ซึ่งไม่ตรงกับกราฟ Fig. 6 ที่อ่านได้ประมาณ 6,800 และ 19,900 tps ข้อความใน Section VIII-B ระบุว่าการยืนยันขั้นสุดท้ายยังไม่ถึงหนึ่งนาที แต่ Table I แสดง 62.96 วินาทีที่ผู้โจมตี 25% และ Table IV ระบุว่าวัดการขยายตัวได้ 2,400 validator ทั้งที่การทดลองที่รายงานมีสูงสุด 1,800 โหนด
4. **การทดลองยังไม่ครอบคลุมเงื่อนไขหลัก:** threat model หลักคือผู้โจมตี 25% แต่กรณีที่ต้องใช้ประมาณ 4,200 โหนดใน 7 shard เพื่อให้ได้ 4,000 tps เป็นเพียงค่าประมาณ เพราะแพลตฟอร์มรับภาระไม่ไหว เครือข่ายเป็นการจำลองที่แบนด์วิดท์และความหน่วงคงที่ทุกลิงก์ ไม่มีความแปรปรวนของเครือข่ายจริง ต่างจากงานรุ่นหลังอย่าง Red Belly [8] ที่ทดลองบนเครื่องเสมือนราว 1,000 เครื่องซึ่งกระจายอยู่หลายประเทศ และทุกตารางไม่รายงานค่าส่วนเบี่ยงเบนหรือจำนวนรอบที่ทำซ้ำ นอกจากนี้เมื่อคำนวณด้วยสมการ (1) การตั้งค่า [4, 1%] ที่ให้ throughput สูงสุดใน Fig. 6 มีโอกาสล้มเหลวราว 5.9×10^{-4} ต่อ shard ขณะที่อีกสามการตั้งค่าอยู่ราว 2×10^{-6} แท่งนั้นจึงไม่ได้อยู่ที่ระดับความปลอดภัยเดียวกัน
5. **ต้นทุนการเปลี่ยน epoch สูง:** RandHound ใช้เวลามากกว่า 70% ของการ bootstrap และกรณีแย่สุดประมาณ 3 ชั่วโมง ระบบจึงต้องใช้ epoch ยาวระดับวัน และผู้เขียนยอมรับว่าไม่เหมาะกับผู้โจมตีที่ปรับตัวเร็ว
6. **ข้อแลกเปลี่ยนด้านความปลอดภัยของการยืนยันเร็วและ liveness:** trust-but-verify ยอมรับโอกาสน้อยกว่า 10% ที่การยืนยันชั้นแรกจะเสี่ยงต่อ double-spending, Atomix พึ่ง client หรือฝ่ายอื่น

ในการปลดล็อก และการวิเคราะห์ความปลอดภัยเป็นแบบ informal โดยไม่วิเคราะห์แรงจูงใจทางเศรษฐศาสตร์ของผู้เข้าร่วม (Section X)

7 ความเห็นและการนำไปประยุกต์ใช้ (Senior Project and Sensor Networks Application)

ความเห็นโดยรวมของผู้จัดทำ: OmniLedger [1] เป็นงานที่ตอบโจทย์ระบบกระจายตัวขนาดใหญ่ได้ครบทั้งทฤษฎีและการสร้างระบบจริง แต่ผลที่ควรนำไปอ้างอิงคือแนวคิดเชิงสถาปัตยกรรมและแบบจำลองความเสี่ยง มากกว่าตัวเลข throughput ซึ่งขึ้นกับรูปแบบธุรกรรมและมีความไม่สอดคล้องกันภายในบทความ

การเชื่อมโยงกับงานวิศวกรรมซอฟต์แวร์ของผู้จัดทำ: การเทียบต่อไปนี้เป็นความเห็นของผู้จัดทำเพื่อใช้อธิบายแนวคิด ไม่ได้อยู่ในบทความ

แนวคิดใน **OmniLedger**	สิ่งที่คล้ายกันในงานระบบแบ็กเอนด์	ความต่างที่ต้องระวัง
shard และการกระจาย UTXO ตาม hash	partition ของ Kafka ที่กระจายข้อความตาม key	Kafka ไม่ได้ออกแบบให้ทนต่อโหนดที่ตั้งใจโกง OmniLedger จึงต้องสุ่มสมาชิก shard ใหม่ทุก epoch
Atomix (Lock แล้ว Unlock to Commit หรือ Abort)	two-phase commit และ Saga pattern ใน microservices	Atomix ให้ client ขับเคลื่อนด้วย proof ที่มีลายเซ็น แทน coordinator กลาง ส่วน Saga ใช้ compensating transaction
state blocks	snapshot, checkpoint และ log compaction	state block ต้องมีลายเซ็นรวมของ shard เพื่อให้โหนดใหม่เชื่อได้โดยไม่ replay
ByzCoinX commit ธุรกรรมที่ไม่ขัดแย้งแบบขนาน	concurrency ด้วย goroutine และการจัดการ conflict ในฐานข้อมูล	ผลทดลองชี้ว่าการขนานมากเกินไปทำให้ประสิทธิภาพลดลง

การประยุกต์กับเครือข่ายเซนเซอร์ (ข้อเสนอของผู้จัดทำ ไม่ใช่ข้อค้นพบของบทความ): OmniLedger ไม่ได้กล่าวถึงเซนเซอร์หรือ IoT แนวคิดที่นำมาใช้ได้อยู่ในระดับ gateway หรือ fog ซึ่งอยู่ระหว่างโหนดเซนเซอร์กับ cloud ในสถาปัตยกรรมเครือข่ายเซนเซอร์แบบหลายชั้น [9] เช่น แบ่ง gateway ตามพื้นที่เป็นกลุ่มคล้าย shard ให้แต่ละกลุ่มยืนยันบันทึกข้อมูลเซนเซอร์ของตน สุ่มสลับสมาชิกกลุ่มเป็นระยะเพื่อลดความเสี่ยงจาก gateway ที่ถูกยึด และใช้แนวคิดแบบ Atomix กับเหตุการณ์ที่ต้องบันทึกพร้อมกันหลายพื้นที่ ข้อจำกัดสำคัญคือโหนดเซนเซอร์มีพลังงาน แบนด์วิดท์ และหน่วยประมวลผลจำกัด ขณะที่ BFT consensus และลายเซ็นดิจิทัลมีต้นทุนการสื่อสารสูง จึงไม่เหมาะจะรันบนโหนดเซนเซอร์โดยตรง

การใช้ในเล่มโครงงาน: บทความนี้ใช้ได้ในบทที่ 2 (ทฤษฎีและงานวิจัยที่เกี่ยวข้อง) หัวข้อการขยายขนาดของระบบกระจายตัวและ Byzantine fault tolerance โดยควรอ้างทั้งแนวคิดหลักและข้อจำกัดในหัวข้อ 6 ควบคู่กัน

เอกสารอ้างอิง (References)

[1] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding," in *Proc. 2018 IEEE Symposium on Security and Privacy (SP)*, San Francisco, CA, USA, 2018, pp. 583–598, doi: 10.1109/SP.2018.000-5.

[2] E. Kokoris-Kogias, P. Jovanovic, N. Gailly, I. Khoffi, L. Gasser, and B. Ford, "Enhancing Bitcoin Security and Performance with Strong Consistency via Collective Signing," in *Proc. 25th USENIX Security Symposium*, 2016.

[3] E. Syta, P. Jovanovic, E. Kokoris-Kogias, N. Gailly, L. Gasser, I. Khoffi, M. J. Fischer, and B. Ford, "Scalable Bias-Resistant Distributed Randomness," in *Proc. 38th IEEE Symposium on Security and Privacy*, 2017.

[4] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A Secure Sharding Protocol for Open Blockchains," in *Proc. 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 17–30.

[5] G. Danezis and S. Meiklejohn, "Centrally Banked Cryptocurrencies," in *Proc. 23rd Network and Distributed System Security Symposium (NDSS)*, 2016.

[6] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.

[7] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proc. 3rd Symposium on Operating Systems Design and Implementation (OSDI)*, 1999, pp. 173–186.

[8] T. Crain, C. Natoli, and V. Gramoli, "Red Belly: A Secure, Fair and Scalable Open Blockchain," in *Proc. 2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 466–483, doi: 10.1109/SP40001.2021.00087.

[9] H. Karl and A. Willig, *Protocols and Architectures for Wireless Sensor Networks*. Chichester, U.K.: Wiley, 2005.